SDEC: Semantic Deep Embedded Clustering

Mohammad Wali Ur Rahman *, Ric Nevarez †, Lamia Tasnim Mim ‡, Salim Hariri * *Senior Member, IEEE*

***Abstract*—The high dimensional and semantically complex nature of textual Big data presents significant challenges for text clustering, which frequently lead to suboptimal groupings when using conventional techniques like k-means or hierarchical clustering. This work presents Semantic Deep Embedded Clustering (SDEC), an unsupervised text clustering framework that combines an improved autoencoder with transformer-based embeddings to overcome these challenges. This novel method preserves semantic relationships during data reconstruction by combining Mean Squared Error (MSE) and Cosine Similarity Loss (CSL) within an autoencoder. Furthermore, a semantic refinement stage that takes advantage of the contextual richness of transformer embeddings is used by SDEC to further improve a clustering layer with soft cluster assignments and distributional loss. The capabilities of SDEC are demonstrated by extensive testing on five benchmark datasets: *AG News, Yahoo! Answers, DBPedia, Reuters 2,* and *Reuters 5*. The framework not only outperformed existing methods with a clustering accuracy of 85.7% on *AG News* and set a new benchmark of 53.63% on *Yahoo! Answers*, but also showed robust performance across other diverse text corpora. These findings highlight the significant improvements in accuracy and semantic comprehension of text data provided by SDEC's advances in unsupervised text clustering.**

***Index Terms*—Clustering, Embeddings, BERT, Autoencoder, Semantic Loss, Distributional Loss, Deep Learning, Natural Language Processing (NLP), Fine-Tuning.**

I. Introduction

The introduction of transformer-based models, like BERT, has significantly advanced the field of natural language processing (NLP) in recent years [1]. These models have established new standards in a number of NLP tasks, such as named entity recognition [2], text classification [3]–[6], and question answering [7], [8]. Text clustering is still a difficult task though, especially when working with big and complicated datasets. Since the ideal number of clusters or the ideal separation of data is frequently ambiguous, clustering is known to be an NP-hard problem, meaning there is no clear-cut solution or answer to the clustering problem [9], [10]. Conventional clustering methods such as k-means and hierarchical clustering often fail to fully capture the semantic nuances of textual data. To address these issues, this paper presents SDEC (Semantic Deep Embedded Clustering), a novel text clustering framework that makes use of transformer embeddings and an autoencoder fine-tuning technique with specialized loss functions.

Tasks like document sorting, topic detection, and information retrieval depend on text clustering, which organizes related texts into coherent clusters. The high dimensionality of text data, capturing semantic similarities, improving clustering accuracy, and guaranteeing scalability are some of the challenges this process must overcome. The curse of dimensionality causes traditional clustering techniques to frequently falter in high-dimensional spaces [11]. Moreover, these conventional algorithms usually depend on Euclidean distances, which may fail to capture the semantic relationships among text data effectively [12].

One major obstacle in textual data processing is the sparsity and noise. High-dimensional text data typically include a lot of irrelevant features, complicating the ability of clustering algorithms to detect meaningful patterns. To mitigate this issue, feature extraction techniques such as Latent Semantic Analysis (LSA) [13] and Term Frequency-Inverse Document Frequency (TF-IDF) [14] have been used. But these methods frequently rely on human feature engineering and often fail to capture deeper semantic relationships.

Word embeddings [15] and, more recently, transformer embeddings [1] have been used in recent methods to provide dense and semantically rich text representations. By capturing contextual information, these embeddings significantly enhance the performance of clustering algorithms. However, the complexity and variability of textual data make it difficult to cluster text data effectively even with these sophisticated embeddings. To address this, specialized techniques are required.

To overcome these text clustering issues, we present the SDEC framework, which introduces several novel features. Specifically, we use BERT to produce dense, semantically rich transformer embeddings that we optimize with a novel mix of pooling algorithms and normalizing techniques to improve text processing. The core of SDEC is a novel semantic loss function that combines Cosine Similarity Loss and Mean Squared Error (MSE). This method increases the accuracy of clustering by using a dynamic weight allocation mechanism for these losses and refined data point assignments, while also maintaining semantic relationships during the reconstruction process of the autoencoder. Furthermore, a refinement stage for semantic clustering improves the quality of the clusters by modifying their assignments according to contextual similarities, making the clusters more accurate representations of the underlying data structure. When combined, these components support a strong clustering performance that successfully handles the

This work was supported in part by the National Science Foundation (NSF) projects under Grant 1624668, Grant 1921485, Grant 2413009 and WISPER Center. Support was also provided by the 2025 Technology and Research Initiative Fund/National Security Systems Initiative administered by the University of Arizona, Office of Research and Partnerships, funded under Proposition 301, the Arizona Sales Tax for Education Act, in 2000. Any opinions, findings, conclusions, or recommendations expressed in this paper are those of the author(s) and do not necessarily reflect the views of NSF.

* Mohammad Wali Ur Rahman and Dr. Salim Hariri are with the Department of Electrical and Computer Engineering from the University of Arizona, Tucson, AZ 85721, USA. Email: {mwrahman, hariri}@arizona.edu.

† Ric Nevarez is with Trustweb, New York, NY 10001, USA. Email: ric@thetrustweb.com.

‡ Lamia Tasnim Mim is with the Department of Computer Science from New Mexico State University, Las Cruces, NM 88003, USA. Email: lamia@nmsu.edu.

nuanced complexity of textual data.

We demonstrate the effectiveness of SDEC through extensive experiments conducted on five benchmark datasets: *AG News*, *Yahoo! Answers*, *DBPedia*, and two *Reuters* datasets. On the *AG News* dataset, SDEC achieved a clustering accuracy of 85.7%, surpassing previous methods and setting a new benchmark. For the more challenging *Yahoo! Answers* dataset, which comprises ten distinct clusters, our system attained a mean clustering accuracy of 53.63%, improving upon the prior best of 52.6%. SDEC also maintained strong performance on *DBPedia, Reuters 2,* and *Reuters 5*, showcasing its generalizability across diverse datasets.

In addition to generalizability, we also address the scalability of our clustering methods. Section II reviews related work in the field, Section III describes the methodology of the SDEC framework, and Section IV details the experimental setup. Section V provides a detailed complexity analysis of the SDEC algorithm, assessing the computational costs during both the training and testing phases. Section VI presents the results, demonstrating the effectiveness of SDEC across various datasets. Finally, Section VII offers conclusions, highlighting the contributions and findings of our research. These sections together underscore SDEC's capability to handle large-scale datasets while achieving high clustering accuracy.

II. Related Works

A. Advances in General Clustering Techniques

Unsupervised learning relies heavily on clustering, and many techniques have been developed to increase the task's scalability and performance. Because of their ease of use and interpretability, traditional clustering algorithms like k-means [16] and hierarchical clustering [17] have been used extensively. Density-based clustering algorithms, such as DBSCAN, HDBSCAN, DenLAC [18]–[20] identify clusters based on local density variations and can detect arbitrarily shaped clusters and effectively managing noise. Nevertheless, these techniques frequently have trouble handling high-dimensional data and might miss intricate patterns within the data.

Researchers have proposed various advanced clustering techniques to address these limitations. For example, before clustering, spectral clustering [21] reduces the dimensionality by using the eigenvalues of a similarity matrix. The PV-DM [22] model learns a fixed vector for each document as a step of dimensionality reduction and combines it with word vectors to predict the next word [23].

Another promising direction is community detection which characterizes communities of similar items by representing data as network rather than purely feature-space distances [24]. Graph-based community detection methods, including modularity maximization algorithms like Louvain [25], partition documents into communities and model them as nodes in a similarity network. The effectiveness of community detection has been further enhanced by approaches that integrate hybrid communities-clustering graph structures [26]. FN-BERT-TFIDF [27] model detects geolocation-content-based communities on Twitter using topic modeling and deep learning for real-time hazard analysis and misinformation detection.

The capacity of deep learning-based clustering techniques to extract complex representations from data has drawn much interest in recent years. Using autoencoder networks and clustering, Deep Embedded Clustering (DEC) [28] learns feature representations and iteratively refines cluster assignments. Dizaji et al. [29] proposed Deep Embedded Regularized Clustering (DEPICT), which employed a convolutional autoencoder (CAE) and stacked a softmax layer on the embedded features to introduce a novel clustering loss. Li et al. [30] presented Discriminatively Boosted Image Clustering (DBC), adopting a self-paced learning strategy where simpler instances are tackled first, then progressively introducing more complex samples. Both approaches build upon the principles of DEC but incorporate additional mechanisms, such as softmax-based clustering in DEPICT and self-paced learning in DBC, to further improve clustering performance. A reconstruction loss is added to DEC in the Improved Deep Embedded Clustering (IDEC) [31] in order to maintain the local structure of the data while clustering.

For clustering tasks, variational autoencoders (VAEs) [32] have also been investigated. In order to enhance clustering performance, VAEs have been coupled with clustering techniques to learn probabilistic latent variable models. One such method that allows for better handling of complex data distributions is the Gaussian Mixture Variational Autoencoder (GMVAE) [33]. It models the latent space as a mixture of Gaussians.

Furthermore, clustering has been done using Generative Adversarial Networks (GANs) [34]. By applying cluster-level constraints to the latent space, ClusterGAN [35] combines clustering with GANs, producing better clustering outcomes.

In order to overcome the limitations of DEC, Lee et al. [36] developed a deep embedded clustering framework for mixed data that handles both numerical and categorical data while enhancing convergence stability through the use of soft-target updates. When applied to mixed data, their method outperforms conventional approaches.

B. Recent Innovations in Text Clustering Approaches

Topic modeling has long been a foundational approach in text analysis, aiming to summarize a larger collection of documents. Classical topic modeling methods such as Latent Dirichlet Allocation (LDA) [37] and Non-negative Matrix Factorization (NMF) [38] have historically played a significant role in text clustering and topic detection. LDA models documents as probabilistic mixtures over latent topics, providing interpretable topic distributions. NMF offers an alternative by decomposing document-term matrices into two non-negative matrices often leading to more coherent topics. Clustering techniques have become integral to various social media analysis tasks, enabling insights into user behavior, event dynamics, and public opinion. For example, deep learning-based clustering architectures have been employed to predict audience interest in emerging news topics on social platforms. These models, either independently or combined with additional techniques such as transformer-based embeddings, have been widely applied to analyze sentiment, reputation, influence, and other social dynamics within topic-specific contexts [39]–[42]. Studies have explored ways to enhance topic modeling

outcomes by comparing different weighting schemes (e.g., TF, TF-IDF) and incorporating contextual cues and document level embedding to create better coherent topics [43]–[45]. However, they rely on bag-of-words assumptions, neglecting deep semantic and contextual information.

The high dimensionality and sparsity of text data make text clustering particularly difficult. Various feature extraction techniques have been used to adapt traditional methods, like k-means and hierarchical clustering, for text data. For example, Latent Semantic Analysis (LSA) [13] and Term Frequency-Inverse Document Frequency (TF-IDF) [14] have been used extensively to translate text into numerical features.

By offering dense and semantically meaningful representations of words, word embeddings like Word2Vec [15], GloVe [46], and FastText [47] have completely changed text clustering. Similar texts can be grouped together more easily thanks to these embeddings, which capture contextual information. Nevertheless, polysemy and capturing sentence context remain difficult tasks for word embeddings.

By offering contextual embeddings that take the entire sentence into account, transformer-based models like BERT [1] have advanced the field even further. Text clustering is one of the many NLP tasks where BERT embeddings have proven effective. Subakti et al. [48] investigated the use of BERT for text clustering, comparing it to more established techniques such as TF-IDF and analyzing its performance as a data representation for text clustering.

Clustering textual data is still challenging despite these developments. According to Subakti et al. [48], although BERT performs better than TF-IDF in the majority of metrics, the effectiveness is heavily reliant on the feature extraction and normalization techniques chosen. Utilizing a range of pooling strategies, including max pooling and mean pooling, and then varying normalization methods, they demonstrated how the clustering algorithm employed can have a substantial impact on how effective BERT representations are.

In order to address joint optimization, their study also made use of DEC and IDEC, which combine deep learning models with clustering layers. However, they only employed Mean Squared Error (MSE) losses for reconstruction, which are less useful for text data. Their detailed study concentrated mainly on comparing BERT with TF-IDF. We get around this drawback by using semantic loss functions, which are more appropriate for identifying semantic similarities in text.

To improve clustering performance on complex text datasets, our proposed framework makes use of transformer embeddings and an autoencoder fine-tuning approach with specialized semantic loss functions. In order to guarantee stable and efficient training, we also included convergence checks during the fine-tuning procedure by continuously monitoring delta values and KL divergence losses. Our approach yields improved semantic representation and clustering accuracy by combining semantic and distributional loss functions, as our benchmark dataset experiments show. By optimizing representation learning and clustering simultaneously, this integrated approach produces more cohesive and semantically significant clusters.

III. Methodology

Our proposed text clustering framework's general architecture makes use of transformer embeddings' strength and autoencoders' ability to fine-tune using specialized semantic loss functions. The goal of this architecture is to generate well-clustered, semantically meaningful representations of text data. Below is a summary of the architecture's main elements and procedures.

Our Semantic Deep Embedded Clustering (SDEC) framework is composed of three main stages: autoencoder-based latent space transformation, clustering with fine-tuning, and semantic refinement. To learn a compressed latent representation, text data is first transformed into BERT embeddings, which are subsequently run through an autoencoder. Through the reduction of noise and the capture of underlying semantic structures, this latent space facilitates more efficient clustering. A combination of distributional and semantic losses is used to further refine the clustering process, ensuring significant topic separations and high intra-cluster similarity. In order to produce clearly defined and semantically coherent clusters, a semantic refinement phase iteratively improves cluster assignments by adjusting cluster centroids based on contextual similarity.

Our framework for semantically enhanced SDEC text clustering with combined semantic and distributional losses fine-tuning is shown in Figure 1.

The preprocessing steps, including text cleaning, lemmatization, and tokenization, as well as the generation of BERT embeddings using different pooling and normalization strategies, are detailed in Appendix A and Appendix B, respectively. These steps ensure that the textual data is transformed into high-quality embeddings suitable for clustering.

A. Autoencoder Training

The autoencoder is an essential part of our text clustering framework. In order to reconstruct the original embeddings from this latent representation, high-dimensional BERT embeddings are compressed into a lower-dimensional latent space. This procedure makes sure that the most important characteristics of the data are kept while minimizing noise, which facilitates efficient clustering.

An encoder and a decoder network comprise the autoencoder configuration. By compressing the input embeddings into a lower-dimensional bottleneck layer, the encoder is able to extract the most important data features. From this compressed representation, the decoder then reconstructs the original embeddings. Thus, it is imperative that the bottleneck layer maintains the information that the embeddings need for precise reconstruction and subsequent clustering.

In order to prevent overfitting, the autoencoder architecture used in this study consists of multiple layers with L2 regularization and SeLU (Scaled Exponential Linear Unit) activation. The selection of SeLU was driven by its self-normalizing characteristic, which aids in preserving stable mean and variance of neuron outputs without significant dependence on supplementary normalization layers. As a result, training remains less susceptible to both vanishing and exploding gradients [49]. This stability, coupled with the controlled output range

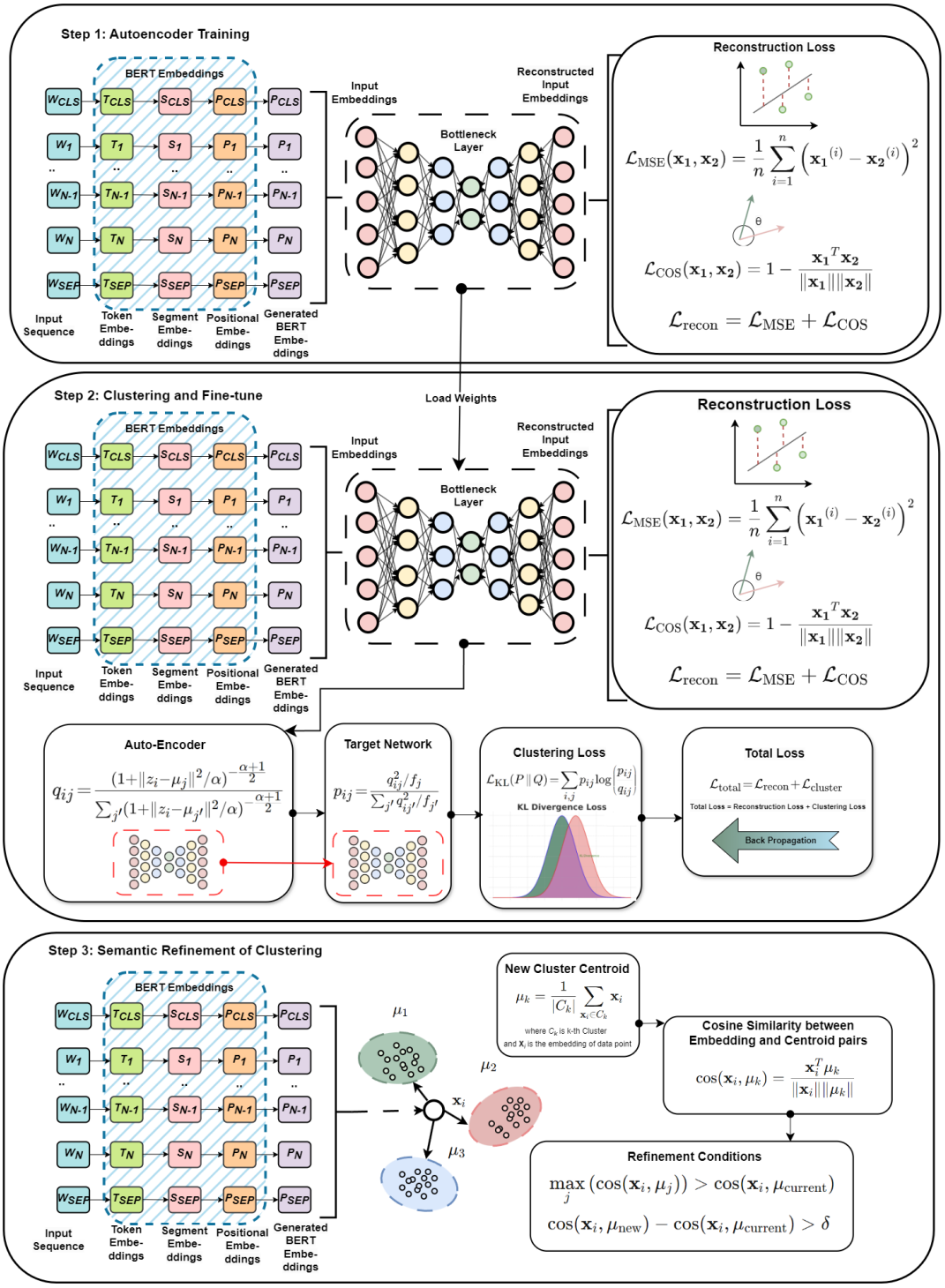


Fig. 1: Semantic Deep Embedded Clustering Framework for Text Data

inherent in SeLU, minimizes large updates in the network parameters, reducing the tendency to memorize noisy patterns. When combined with L2 regularization, which directly penalizes overly large weights. SeLU thus encourages the network to focus on preserving the most salient characteristics of the input embeddings rather than learning dataset-specific patterns, ultimately yielding more robust latent representations for clustering.

The input data is compressed by the encoder using several layers, and the original input is restored by the decoder by mirroring this process. The encoder network reduces the dimensionality of the input embeddings through successive layers with SeLU activation and L2 regularization. The final layer of the encoder is the bottleneck layer, which captures the most salient features of the input data.

The bottleneck layer serves as the most compressed repre-

sentation of the input data. It is defined as follows:

$$\mathbf{h} = f(\mathbf{W}_h\mathbf{x} + \mathbf{b}_h), \tag{1}$$

where $\mathbf{x}$ represents the input embeddings, $\mathbf{W}_h$ and $\mathbf{b}_h$ are the weights and biases of the bottleneck layer, and f is the activation function.

Once the bottleneck representation is obtained, the decoder network reconstructs the original embeddings. The output layer of the decoder has a linear activation function to match the dimensionality of the input embeddings.

To improve reconstruction accuracy, a combined loss function is employed that integrates both Mean Squared Error (MSE) and Cosine Similarity Loss. This ensures that the reconstruction preserves both the magnitude (via MSE) and the semantic similarity (via cosine similarity) of the original embeddings. The Mean Squared Error (MSE) Loss measures the average squared difference between the original embeddings $\mathbf{x}_i$ and the reconstructed embeddings $\hat{\mathbf{x}}_i$, defined as:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(\mathbf{x}_i - \hat{\mathbf{x}}_i)^2 \tag{2}$$

Additionally, the Cosine Similarity Loss measures the cosine of the angle between the original and reconstructed embeddings, ensuring that they retain their semantic similarity. It is given by:

$$\cos(\mathbf{x}_i, \hat{\mathbf{x}}_i) = \frac{\mathbf{x}_i \cdot \hat{\mathbf{x}}_i}{\|\mathbf{x}_i\|\|\hat{\mathbf{x}}_i\|} \tag{3}$$

where $\mathbf{x}_i \cdot \hat{\mathbf{x}}_i$ denotes the dot product, and $\|\mathbf{x}_i\|$ and $\|\hat{\mathbf{x}}_i\|$ denote the magnitudes of the vectors. The Cosine Similarity Loss is then given by:

$$\mathcal{L}_{\text{cosine}} = 1 - \cos(\mathbf{x}_i, \hat{\mathbf{x}}_i) \tag{4}$$

The twofold nature of textual embedding preservation is addressed by combining MSE and Cosine Similarity Loss. In particular, MSE enforces accuracy in each embedding dimension, ensuring the reconstructed vectors retain their numerical scale and overall magnitude. In contrast, Cosine Similarity Loss preserves the angular alignment between the original and reconstructed vectors, an essential characteristic for embeddings in natural language tasks, where direction often encodes semantic relationships (e.g., topic or sentiment). Relying on MSE alone could preserve magnitude but allow semantic drifts; conversely, using only cosine similarity could yield consistent directions but overlook scale-related nuances that can be meaningful in certain embedding dimensions.

The synergy between these two goals is further enhanced by the dynamic weighting mechanism. The MSE loss is given a correspondingly higher weight at each training step if the autoencoder has a harder time matching the scale of the embeddings (higher MSE). On the other hand, the model allocates more capacity to directional vector alignment if maintaining the semantic orientation (i.e., cosine similarity) turns into the bottleneck. The network won't overcommit to one loss component while neglecting the other thanks to this adaptive emphasis. Rather, it responds to the current reconstruction difficulty by continuously balancing the two goals, which speeds up convergence and improves final embeddings.

To optimize the model effectively, the weights between these two losses are dynamically adjusted based on their relative magnitudes, ensuring that the model focuses on whichever loss term is more dominant during training:

$$\mathcal{L}_{\text{recon}} = \mathit{weight}_{\text{MSE}} \times \mathcal{L}_{\text{MSE}} + \mathit{weight}_{\text{cosine}} \times \mathcal{L}_{\text{cosine}} \tag{5}$$

where the weights are computed as:

$$\mathit{weight}_{\text{MSE}} = \frac{\mathcal{L}_{\text{MSE}}}{\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{cosine}} + \epsilon} \tag{6}$$

and

$$\mathit{weight}_{\text{cosine}} = \frac{\mathcal{L}_{\text{cosine}}}{\mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{cosine}} + \epsilon} \tag{7}$$

where ϵ is a small constant to prevent division by zero. This combined semantic loss function ensures that the reconstructed embeddings maintain both semantic alignment (through cosine similarity) and their original magnitude and scale (through MSE). The dynamic weight allocation allows the model to adaptively focus on the more challenging aspect of reconstruction during training.

The autoencoder is fitted to the training set of data during the training phase, and its performance is verified on a separate test set. The goal is to make sure the autoencoder learns a concise and useful representation of the input data, as this representation can be used for clustering.

The ability of an autoencoder to learn a lower-dimensional representation of high-dimensional data makes it important for clustering tasks. This representation reduces noise and makes clustering easier by capturing the data's key characteristics. More specifically, for accurate clustering and reconstruction, the bottleneck layer provides a compact representation that retains the most important data.

By employing the autoencoder, we ensure the effective compression and reconstruction of the BERT embeddings, providing a solid foundation for the subsequent clustering phases. As such, the autoencoder plays a crucial role in enhancing the accuracy and performance of our text clustering framework.

B. Clustering Layer

Our framework's clustering layer transforms the input sample (feature) into a soft label that indicates the likelihood that the sample will fall into each cluster. The similarity between the embedded point and the cluster center determines this probability. By integrating the clustering process within the neural network, both clustering and representation learning are optimized simultaneously, ensuring more cohesive and semantically meaningful clusters. Figure 2 shows the SDEC network structure with the cluster layer. The clustering layer outputs a probability distribution (soft labels) across all clusters for each input sample. Hard cluster labels (y_i) are subsequently assigned by taking the $\arg\max$ of these probabilities.

A customized Keras layer is used to implement this clustering mechanism. This clustering layer acts as a soft-assignment layer based on the Student's t-distribution. During training, it computes the soft assignments for every sample, initializes the cluster centers, and updates them as the model learns. The

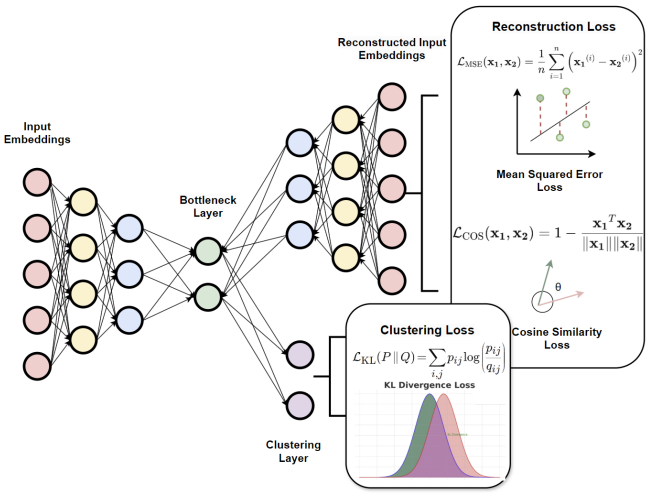


Fig. 2: SDEC Network Structure with Cluster Layer

clustering layer initializes the cluster centers μ_j using the K-means++ procedure, ensuring that the initial centers are well-distributed in the latent space [16].

For each input sample $\mathbf{z}_i$ in the latent space, the soft assignment q_{ij} to cluster j is calculated using the Student's t-distribution as a kernel:

$$q_{ij} = \frac{(1 + \frac{\|\mathbf{z}_i - \mu_j\|^2}{\alpha})^{-\frac{\alpha+1}{2}}}{\sum_{j'}(1 + \frac{\|\mathbf{z}_i - \mu_{j'}\|^2}{\alpha})^{-\frac{\alpha+1}{2}}}, \tag{8}$$

where α is the degrees of freedom of the Student's t-distribution, $\mathbf{z}_i$ is the latent representation of sample i, and μ_j is the cluster center j [50]. The target distribution p_{ij} is then computed to enhance cluster purity by emphasizing high-confidence assignments:

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'}(q_{ij'}^2 / \sum_i q_{ij'})}, \tag{9}$$

where q_{ij} is the soft assignment from the previous step [36].

The clustering layer employs the Kullback-Leibler (KL) divergence loss to measure the difference between the target distribution p and the predicted distribution q:

$$\mathcal{L}_{\text{KL}} = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}, \tag{10}$$

where p_{ij} is the target distribution and q_{ij} is the predicted distribution [32].

Final hard cluster assignments for each data sample are determined by selecting the cluster index with the highest soft assignment probability:

$$y_i = \arg\max_j q_{ij} \tag{11}$$

For optimization, the model uses Stochastic Gradient Descent (SGD) with a learning rate of 0.01 and momentum of 0.9 [51]. The total loss for the entire network is defined as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{cluster}} + \mathcal{L}_{\text{recon}}, \tag{12}$$

where $\mathcal{L}_{\text{recon}}$ is the combined semantic loss from the reconstruction process, and $\mathcal{L}_{\text{cluster}}$ corresponds to the KL divergence loss from the clustering layer.

Soft target updates are performed by iteratively updating the target distribution p based on the current soft assignments q. This strategy refines the clustering results and improves cluster purity. During training, an iterative process updates both the model parameters via the combined loss function and the cluster centers using the predicted soft assignments. Convergence is tracked using the KL divergence and delta label changes, ensuring stability and efficiency in the clustering process.

Consistent with standard deep clustering literature, the number of clusters k is treated as a priori knowledge and corresponds directly to the known number of categories in each dataset. If the true number of clusters is unknown, it can be estimated using methods such as the elbow method, silhouette analysis, or eigengap heuristics [52].

Our method incorporates the clustering layer into the same neural network used for representation learning, ultimately enabling the reconstruction objective and the clustering objective to reinforce one another. By carefully combining these processes within a single framework, we obtain more cohesive and semantically meaningful clusters, as the network can capture both the essential data features and their underlying group structures.

In practice, simultaneously training the autoencoder and the clustering layer can lead to competing gradients, where the reconstruction objective seeks to retain as much information as possible while the clustering objective forces separation of latent representations. Training them together often causes numeric instability or forces premature convergence to sub-optimal cluster centroids. By first allowing the autoencoder to learn a stable low-dimensional representation, we ensure that subsequent clustering is performed on embeddings that accurately capture semantic relationships. By using a phased approach, conflicting optimization signals are avoided and more consistent and reliable clustering results are obtained.

C. Semantic Refinement of Clustering

To improve the coherence of cluster assignments based on semantic similarity, the SDEC (Semantic Deep Embedded Clustering) algorithm has a refinement phase. In the refinement step, data points are reassigned to clusters if their similarity to a new cluster centroid is greater than their current clusters. This technique guarantees that semantically related points are grouped together and enhances cluster purity.

Let $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$ be the set of BERT embeddings, where $\mathbf{x}_i \in \mathbb{R}^d$ is the d-dimensional embedding of the i-th data point, and $C = \{c_1, c_2, \ldots, c_k\}$ be the set of k cluster centroids, with initial labels assigned as $\mathbf{y} = \{y_1, y_2, \ldots, y_n\}$. The refinement process consists of the following steps:

1) Computation of Cluster Centroids

For each cluster c_j, we compute the cluster centroid $\mu_j \in \mathbb{R}^d$, which is the mean of all embeddings that belong to cluster c_j. Mathematically, the centroid μ_j is given by:

$$\mu_j = \frac{1}{|C_j|} \sum_{\mathbf{x}_i \in C_j} \mathbf{x}_i \tag{13}$$

where $|C_j|$ is the number of points in cluster c_j, and $\mathbf{x}_i \in C_j$ represents the data points assigned to cluster c_j.

2) Semantic Similarity and Reassignment

For each point $\mathbf{x}_i$, its similarity to its current cluster centroid μ_{y_i} is computed using cosine similarity:

$$\text{Sim}(\mathbf{x}_i, \mu_{y_i}) = \frac{\mathbf{x}_i \cdot \mu_{y_i}}{\|\mathbf{x}_i\| \|\mu_{y_i}\|} \tag{14}$$

We also compute the cosine similarities of $\mathbf{x}_i$ with all other cluster centroids μ_j, for $j = 1, \ldots, k$. The point is reassigned to a new cluster c_{j^*} if the similarity to the new centroid μ_{j^*} exceeds the similarity to the current centroid by a predefined threshold λ:

$$j^* = \arg\max_j \text{Sim}(\mathbf{x}_i, \mu_j) \tag{15}$$

The point $\mathbf{x}_i$ is reassigned to c_{j^*} if:

$$\text{Sim}(\mathbf{x}_i, \mu_{j^*}) - \text{Sim}(\mathbf{x}_i, \mu_{y_i}) > \lambda \tag{16}$$

The algorithm refines the initial cluster labels y_i to new labels y_i^{new} based on the computed similarities and the threshold λ. The refined labels y_i^{new} are used to improve cluster consistency by grouping semantically similar points together.

The threshold λ acts as a sensitivity parameter that determines how large the similarity difference needs to be for a point to be reassigned. A higher threshold results in fewer reassignments, whereas a lower threshold leads to more aggressive refinement.

When working with high-dimensional embeddings, like BERT embeddings, this refinement technique is especially beneficial because the semantic structure of the data may not be adequately represented by the initial clustering methods. We can make sure that points are assigned to the most semantically appropriate cluster by refining clusters based on semantic similarity.

This robust method works well for high-dimensional feature spaces because cosine similarity is used to guarantee that the refinement is independent of the size of the embeddings and only considers their directional alignment.

IV. Experiments

A. Exploratory Data Analysis

This subsection outlines the datasets utilized in our clustering experiments. The choice of datasets was guided by the need for diversity in topic coverage and data scale, ranging from thousands to over a million samples, to ensure robust testing of the SDEC framework.

1) AG News Dataset

The *AG News* dataset [53] consists of news articles divided into four major categories: World, Sports, Business, and Sci/Tech. We used 5,600 articles for training and 120,000 for testing, maintaining balance across categories to ensure fairness in clustering evaluations.

2) Yahoo! Answers Dataset

The *Yahoo! Answers* dataset [54], with its diverse array of user-generated questions and answers, spans ten different categories. This dataset provides a substantial challenge due to its size and variety, with 60,000 pairs for training and 1,400,000 for testing.

3) DBPedia Dataset

Derived from Wikipedia, the *DBPedia* dataset [55] categorizes structured data into fourteen clusters, ranging from geographical entities to cultural works. We employed 70,000 samples for training and 560,000 for testing.

4) Reuters Dataset

We utilized two subsets of the *Reuters-21578* corpus, focusing on major business and economic categories. The *Reuters 2* subset includes categories *"earn"* and *"acq"*, with 1,243 training and 4,972 testing articles. The more complex *Reuters 5* subset expands to include five major categories including *"earn", "acq", "money-fx", "grain"* and *"crude"*, with 1,442 training and 6,487 testing articles. Notably, the *Reuters 5* dataset is heavily imbalanced, with the *"earn"* and *"acq"* categories comprising the vast majority of articles [56].

These datasets, detailed in Table I, were preprocessed to remove non-textual elements and ensure clean text for clustering. The variety and scale of these datasets are critical for assessing the generalizability and effectiveness of clustering algorithms.

TABLE I: Description of Datasets

Dataset	Train Samples	Test Samples	Clusters	Max/Min Cluster Size Ratio	Clustering Complexity
AG News	5,600	120,000	4	1 : 1	Moderate
Yahoo! Answers	60,000	1,400,000	10	1 : 1	High
DBPedia	70,000	560,000	14	1 : 1	High
Reuters 2	1,243	4,972	2	1.7 : 1	Low
Reuters 5	1,442	6,487	5	13.4 : 1	High

B. Experiment Settings

To provide clarity on our experimental methodology, all hyperparameter settings and training configurations, including details on autoencoder architecture, clustering fine-tuning, and cluster refinement strategies, are provided in Appendix C. These settings define the training conditions under which our SDEC clustering framework was evaluated.

C. Evaluation Metrics

In this subsection, we discuss the evaluation metrics used to assess the performance of our clustering model. The primary metrics considered are Clustering Accuracy (ACC), Normalized Mutual Information (NMI), and Adjusted Rand Index (ARI). Each of these metrics provides a different perspective on the quality of the clustering results, and their mathematical definitions are provided below.

1) Clustering Accuracy (ACC)

Clustering Accuracy (ACC) measures the percentage of correctly assigned labels after the best mapping between the predicted cluster labels and the true labels is found. It is defined as follows:

$$\text{ACC} = \max_m \frac{\sum_{i=1}^{n} \mathbf{1}\{y_i = m(c_i)\}}{n} \tag{17}$$

where y_i is the true label, c_i is the cluster label, m ranges over all possible one-to-one mappings between clusters and labels, $\mathbf{1}\{\cdot\}$ is the indicator function, and n is the total number

TABLE II: Execution Time and Scalability Metrics for SDEC

Dataset	Training Samples	Training Time (s)	Testing Samples	Testing Time (s)	No. of Clusters	Avg. No. of BERT Tokens Combined	Avg. No. of BERT Tokens Training	Avg. No. of BERT Tokens Testing
AG News	5000	1647.22	120000	809.55	4	44.8	44.81	44.59
Yahoo! Answers	60000	2132.05	1400000	1407.7	10	145.64	146.02	144.89
Reuters 2	1243	1151.113	4972	571.19	2	111.56	111.77	109.68
Reuters 5	1442	1339.4	6487	849.58	5	126.51	126.73	124.56
DBPedia	70000	3121.6	560000	1487.83	14	72.37	72.23	72.64

of samples. The accuracy measures how well the clustering model can reproduce the true labels after the optimal mapping.

2) Normalized Mutual Information (NMI)

Normalized Mutual Information (NMI) measures the amount of shared information between the clustering results and the true labels. It normalizes the mutual information score to scale between 0 and 1. The NMI is defined as:

$$\text{NMI}(Y, C) = \frac{2 \cdot I(Y; C)}{H(Y) + H(C)} \tag{18}$$

where $I(Y; C)$ is the mutual information between the true labels Y and the cluster assignments C, and $H(Y)$ and $H(C)$ are the entropies of Y and C respectively. NMI evaluates how much information about the true labels is captured by the cluster assignments, with 1 indicating perfect correlation and 0 indicating no correlation.

3) Adjusted Rand Index (ARI)

The Adjusted Rand Index (ARI) measures the similarity between the predicted and true cluster assignments, adjusting for chance grouping. The ARI is defined as:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \tag{19}$$

where RI is the Rand Index, which calculates the number of pairs of samples that are either correctly assigned to the same cluster or correctly assigned to different clusters, and $\mathbb{E}[\text{RI}]$ is the expected value of the Rand Index for random cluster assignments. ARI ranges from -1 to 1, where 1 indicates perfect agreement, 0 indicates random labeling, and negative values indicate worse-than-random labeling.

By taking into account both the agreement with true labels and the accuracy of cluster assignments, these metrics collectively offer a thorough assessment of the clustering performance.

V. Computational Complexity

In this section, we analyze the computational complexity of the proposed SDEC framework, which consists of three primary components: the autoencoder with a combined loss (MSE and cosine similarity), the clustering process and the semantic refinement. The complexity is presented for both the training and testing phases.

Preprocessing and converting text to BERT embeddings are preparatory steps essential for the subsequent stages of the framework. While these steps have computational costs, they are generally executed once per dataset and do not scale with the core parameters of the clustering algorithm. Therefore, their complexity is not detailed here but is acknowledged to be part of the preprocessing overhead.

A. Training Complexity

The training phase of SDEC involves the autoencoder's forward and backward passes, combined loss computation, clustering, and semantic refinement. The overall complexity is derived by considering the following components:

1) Autoencoder with Combined Loss

The forward and backward propagation through the autoencoder, which consists of L layers, has a complexity of:

$$O\left(2 \times \sum_{i=1}^{L} n_{i-1} \times n_i\right)$$

where n_i represents the number of neurons in the i-th layer.

For the combined loss:

- **Mean Squared Error (MSE) loss**: The complexity of MSE loss is $O(B \times d)$, where B is the batch size and d is the dimensionality of the embeddings.
- **Cosine Similarity loss**: The complexity of the cosine similarity loss is also $O(B \times d)$.

Hence, the total complexity of the autoencoder with the combined loss function becomes:

$$O\left(2 \times \sum_{i=1}^{L} n_{i-1} \times n_i + B \times d\right)$$

2) Clustering and Semantic Refinement

The clustering phase in SDEC includes:

- **K-means initialization**: This has a complexity of $O(n_b \times k \times d)$, where n_b is the number of data points, k is the number of clusters, and d is the dimensionality of the embeddings
- **KL Divergence loss**: Calculating the KL divergence loss between the soft cluster assignments has a complexity of $O(i \times n_b \times k \times d)$ where i represents the number of iterations for clustering convergence.
- **Cosine Similarity for cluster refinement**: The semantic refinement process based on cosine similarity also contributes $O(n_b \times k \times d)$.

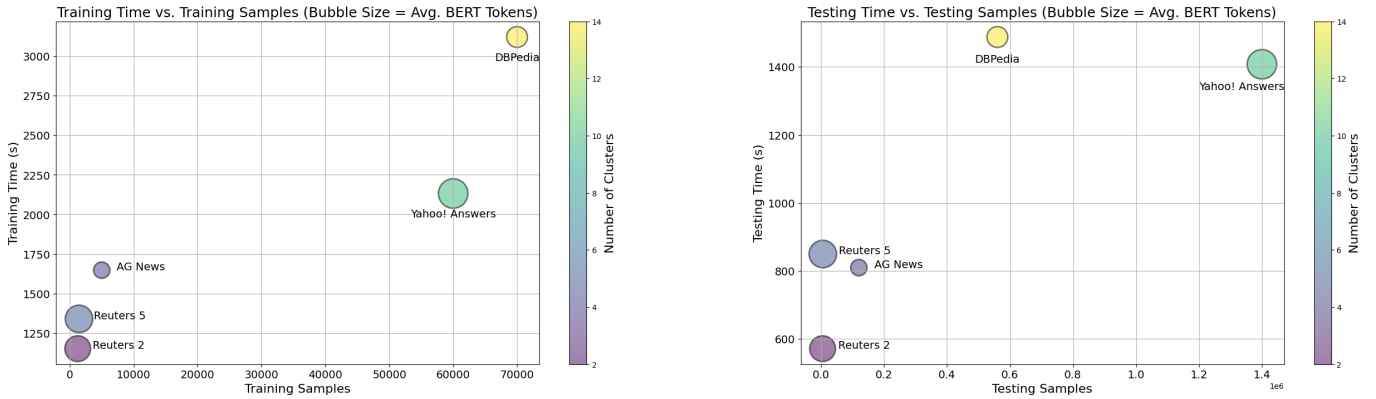
Thus, the total complexity for clustering and refinement becomes:

$$O(n_b \times k \times d + i \times n_b \times k \times d)$$

3) Total Training Complexity

The total computational complexity for the training phase, combining both the autoencoder and the clustering process, is:

$$O\left(E \times \left(\sum_{i=1}^{L} n_{i-1} \times n_i + B \times d + n_b \times k \times d \times (i+1)\right)\right)$$



(a) Training Time vs. Training Samples (b) Testing Time vs. Testing Samples

Fig. 3: Scalability of the SDEC Algorithm. Bubble size corresponds to the average number of BERT tokens, and color indicates the number of clusters.

where E is the number of epochs, B is the batch size, n_b is the number of data points, k is the number of clusters, and i is the number of clustering iterations.

B. Testing Complexity

In the testing phase, we do not compute backpropagation or the loss functions. The testing complexity consists of the following components:

1) Autoencoder Forward Pass

The forward pass through the autoencoder has a complexity of:

$$O\left(\sum_{i=1}^{L} n_{i-1} \times n_i\right)$$

2) Clustering Assignments

During the testing phase, only the clustering assignments are computed. The complexity of assigning points to clusters based on cosine similarity with cluster centroids is:

$$O(n_b \times k \times d)$$

3) Total Testing Complexity

Thus, the total computational complexity during testing is:

$$O\left(\sum_{i=1}^{L} n_{i-1} \times n_i + n_b \times k \times d\right)$$

The total complexity of SDEC during training is dominated by the autoencoder training, the combined loss calculation, and the clustering with semantic refinement. During testing, the complexity is significantly reduced as only the forward pass and clustering assignments are computed. The overall complexity is scalable and efficient for large-scale text clustering tasks.

C. Execution Time and Scalability of SDEC

This section presents an analysis of the SDEC (Semantic Deep Embedded Clustering) algorithm's scalability and execution time on various datasets. The number of samples, training time, testing time, and average number of BERT tokens processed during training and testing for different datasets are summarized in Table II. Additionally, bubble charts in Figure 3 show the average number of BERT tokens and clusters for each dataset, as well as the relationship between training/testing time and sample sizes.

The scalability of the SDEC algorithm in relation to the number of samples and execution time is demonstrated by the bubble charts. The average number of BERT tokens per dataset is represented by the size of the bubbles, and the number of clusters is indicated by their color.

As shown in the table and visualized in the left chart of Figure 3, the training time increases with the number of training samples across all datasets. *DBPedia* and *Yahoo! Answers*, which have the highest number of training samples (70,000 and 60,000, respectively), also require the longest training times. This is consistent with the linear dependence of training complexity on the number of samples, as derived in the complexity analysis. The bubble sizes indicate that *DBPedia* and *Yahoo! Answers* also process the most BERT tokens, which further contributes to the increased training time.

Similar trends are observed for testing time in the right chart of Figure 3. *DBPedia* and *Yahoo! Answers*, with the largest testing samples (560,000 and 1,400,000, respectively), have the highest testing times. Smaller datasets like *Reuters 2* and *Reuters 5* exhibit significantly shorter testing times, which is expected given their much smaller number of samples. The average number of BERT tokens per dataset also influences testing time, as seen by the varying bubble sizes.

Execution time is also dependent on the number of clusters. Larger datasets with many clusters, 14 for *DBPedia* and 10 for *Yahoo! Answers*, take longer to process than smaller datasets with only two clusters, like *Reuters 2*. The scalability analysis demonstrates the flexibility of the SDEC algorithm in practical applications by indicating that it can handle a broad range of datasets with different sample sizes, cluster counts, and BERT token lengths.

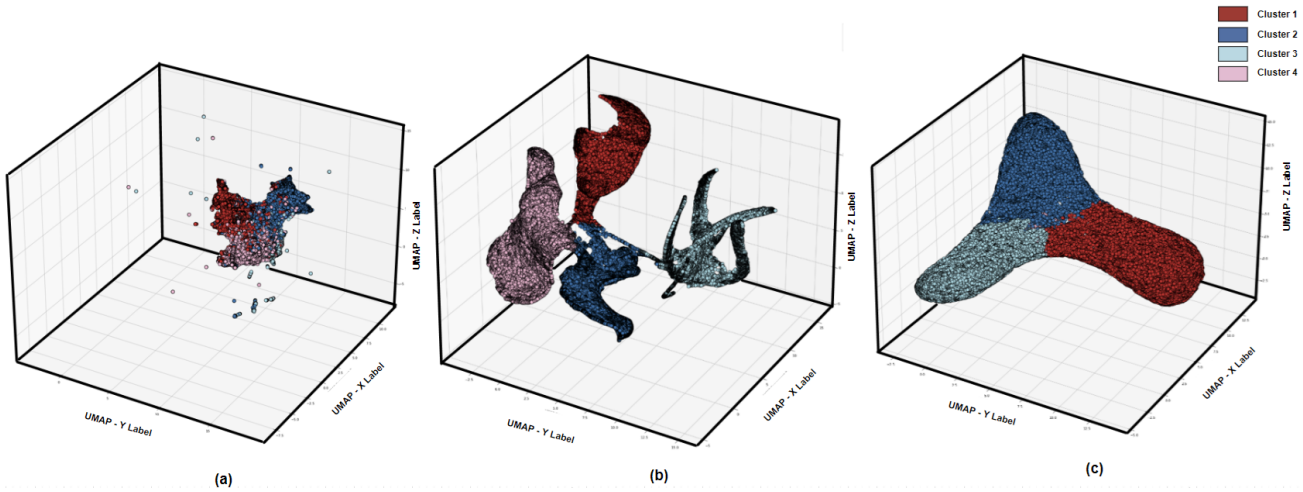


Fig. 4: Visual Illustration of Step-by-Step Improvements in Clustering Performance

VI. Results

A. Step-by-Step Clustering Improvements

In this subsection, we present outcomes from three distinct experimental setups on a randomly selected run of the *AG News* dataset. To emphasize the improvements made through various stages of our methodology, the results are presented through evaluation metrics and visualizations.

Figure 4 shows the clustering results in three stages: (a) initial clustering using an autoencoder with K-Means and MSE as the reconstruction loss, (b) clustering with a DEC layer using MSE for reconstruction and KL divergence for clustering loss without fine-tuning, and (c) our proposed SDEC clustering system with enhanced semantic loss (Cosine Similarity Loss + MSE), fine-tuning of the entire network for improved cluster assignments, and further refinement of clustering based on semantic contextual information.

We note that there is little discernible separation between the clusters in the first stage (Figure 4 (a)) where the data samples are mainly mixed together. The assessment metrics, which show the adjusted Rand index (ARI) at 43.2, normalized mutual information (NMI) at 50.5, and accuracy (ACC) at 56.34, are indicative of this [57]. The autoencoder's use of MSE as the reconstruction loss and the clustering algorithm K-Means fail to adequately capture the underlying structure of the data.

When the DEC layer is added, we observe some improvements in the second stage (Figure 4 (b)). The graph's middle region is one area where the clustering assignments are still unclear, despite the fact that the clusters are now more separated than they were previously. This stage produces an accuracy (ACC) of 69.71, an NMI of 53.5, and an ARI of 50.2. Although there is a notable improvement, the clustering results still leave room for further enhancement.

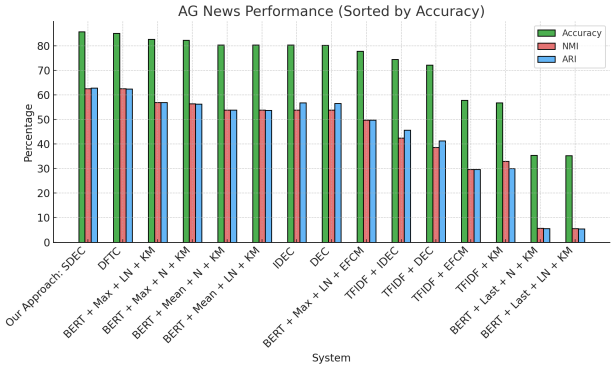
Using a combination of MSE and Cosine Similarity Loss with dynamic weight allocation, our suggested SDEC clustering system achieves notable improvements in the final stage (Figure 4 (c)). The preservation of the embeddings' magnitude and semantic similarity is guaranteed by this combined loss throughout the reconstruction process. Furthermore, by reassigning points based on their similarity to cluster centroids, the semantic refinement step enhances cluster assignments even more. This stage attains an accuracy (ACC) of 82.26, an NMI of 53.04, and an ARI of 56.5. There is very little visual overlap between different clusters, indicating a much stronger visual separation between them. Since Cluster 4 is situated on the other side of the 3D graph, it is not visible in this particular view.

To ensure a fair comparison, the UMAP projections into 3D space for visualization were performed using the same parameters at every stage. The significant enhancement in clustering performance, as observed through numerical and visual means, demonstrates the effectiveness of our suggested methodology.

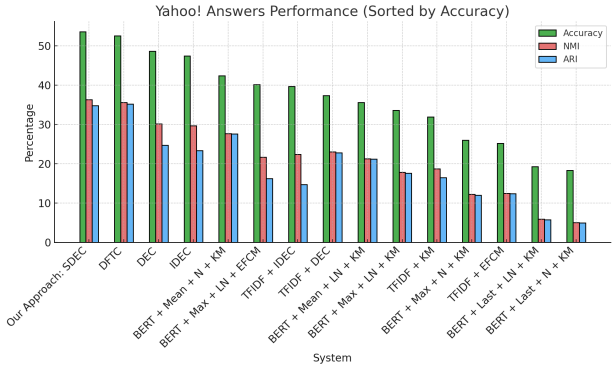
B. Results from Comparative Analysis

In this subsection, we present a comparative analysis of our SDEC clustering system with other systems across multiple datasets, including *AG News, Yahoo! Answers, DBPedia, Reuters 2, and Reuters 5*. As noted in previous studies, many clustering approaches either do not specify the number of samples used or employ relatively small subsets of the datasets, which limits their applicability to large-scale, real-world clustering tasks. For example, studies such as Subakti et al. [48] used only 4000 samples from *AG News* and 10,000 samples from *Yahoo! Answers*. On the other hand, our method makes use of much larger subsets of these datasets, guaranteeing a more comprehensive and reliable assessment.

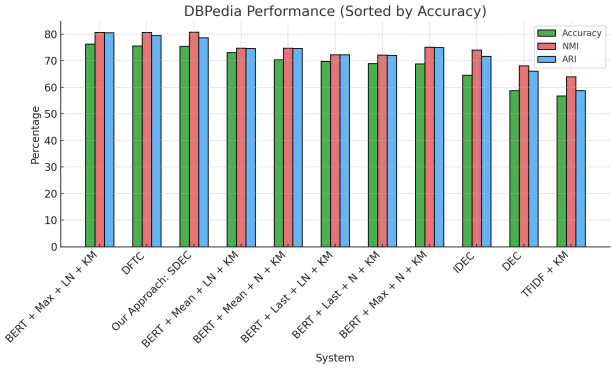
Furthermore, some of the experimental results presented in this analysis were adapted from studies conducted by Subakti et al. and Guan et al. [48], [58]. For the *AG News* and *Yahoo! Answers* datasets, however, we conducted our own experiments for the DEC and IDEC approaches, which demonstrated improvements over the results reported by the aforementioned authors. Therefore, we have included our results in the comparison table specifically for these two datasets. Certain approaches were not tested for the *DBPedia, Reuters 2,* and *Reuters 5* datasets, leading to empty entries in



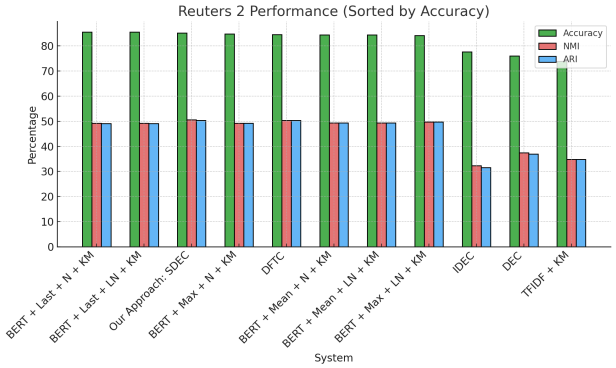
(a) AG News Performance (Sorted by Accuracy)



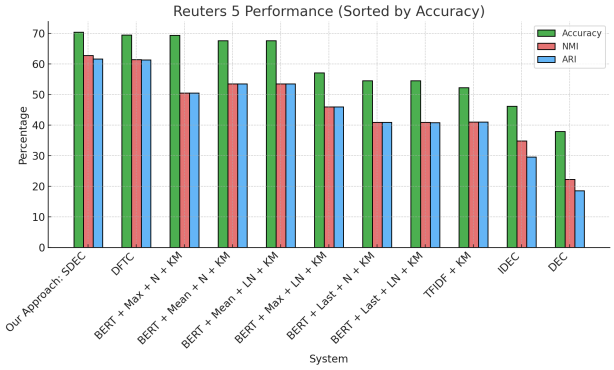
(b) Yahoo! Answers Performance (Sorted by Accuracy)



(c) DBPedia Performance (Sorted by Accuracy)



(d) Reuters 2 Performance (Sorted by Accuracy)



(e) Reuters 5 Performance (Sorted by Accuracy)

Fig. 5: Clustering performance comparison across different datasets.

the corresponding sections of our comparative results table in Table III.

Moreover, other clustering techniques, like those explored by Agarap [59], used semi-supervised methods by augmenting embeddings with labeled training data, thus disqualifying them from direct comparison with our fully unsupervised methodology. Consequently, such systems were excluded from our evaluation. However, we included other relevant studies such as Subakti et al. [48], which used unsupervised approaches like DEC, IDEC, and other K-Means-based clustering techniques, albeit on smaller subsets. Each experiment was run 20 times with random initializations to guarantee the accuracy and robustness of our findings, and the best clustering results were reported.

In the comparison:

- **LSA** refers to Latent Semantic Analysis
- **NMF** refers to Non-negative Matrix Factorization
- **DEC** refers to Deep Embedded Clustering
- **IDEC** refers to Improved Deep Embedded Clustering
- **DFTC** refers to Deep Feature Based Text Clustering
- **TFIDF** refers to Term Frequency Inverse Data Frequency
- **N** refers to standard normalization techniques applied to embeddings.
- **LN** refers to Layer normalization techniques applied to

TABLE III: Clustering Results Across Datasets (Best Results in Bold)

System	AG News			Yahoo! Answers			DBPedia			Reuters 2			Reuters 5		
	Accuracy	NMI	ARI	Accuracy	NMI	ARI	Accuracy	NMI	ARI	Accuracy	NMI	ARI	Accuracy	NMI	ARI
NMF + TFIDF + KM	0.301	0.022	0.006	0.305	0.180	0.081	0.513	0.584	0.246	0.734	0.342	0.214	0.435	0.303	0.031
LSA + TFIDF + KM	0.301	0.023	0.006	0.344	0.102	0.207	0.492	0.589	0.224	0.731	0.339	0.209	0.437	0.303	0.027
LSA + TF-IDF + Spherical KM	0.391	0.103	0.115	0.417	0.226	0.117	0.648	0.666	0.542	0.749	0.173	0.246	0.618	0.372	0.361
NMF + TF-IDF + Spherical KM	0.423	0.125	0.128	0.33	0.188	0.137	0.626	0.646	0.516	0.777	0.252	0.308	0.605	0.348	0.270
TF-IDF + Spherical KM	0.670	0.482	0.482	0.436	0.294	0.215	0.575	0.699	0.515	0.764	0.380	0.277	0.645	0.516	0.398
TFIDF + KM	0.568	0.33	0.30	0.319	0.187	0.165	0.567	0.64	0.588	0.739	0.348	0.348	0.522	0.41	0.41
TFIDF + EFCM	0.578	0.297	0.297	0.252	0.125	0.1236	-	-	-	-	-	-	-	-	-
TFIDF + DEC	0.7211	0.386	0.413	0.374	0.2302	0.228	-	-	-	-	-	-	-	-	-
TFIDF + IDEC	0.745	0.425	0.457	0.397	0.224	0.147	-	-	-	-	-	-	-	-	-
DEC	0.8019	0.538	0.565	0.487	0.3019	0.247	0.588	0.681	0.661	0.76	0.374	0.369	0.379	0.222	0.185
IDEC	0.8038	0.538	0.568	0.475	0.297	0.2339	0.646	0.74	0.717	0.777	0.323	0.315	0.461	0.348	0.295
BERT + Last + N + KM	0.354	0.057	0.056	0.183	0.05	0.049	0.69	0.721	0.72	**0.856**	0.492	0.491	0.545	0.409	0.409
BERT + Last + LN + KM	0.353	0.056	0.055	0.193	0.059	0.057	0.698	0.723	0.723	**0.856**	0.492	0.491	0.545	0.409	0.408
BERT + Max + LN + KM	0.827	0.569	0.569	0.336	0.178	0.176	**0.763**	0.806	0.805	0.842	0.497	0.497	0.571	0.459	0.459
BERT + Max + N + KM	0.823	0.564	0.563	0.26	0.122	0.12	0.688	0.751	0.75	0.848	0.492	0.492	0.693	0.505	0.505
BERT + Mean + N + KM	0.804	0.539	0.538	0.424	0.277	0.276	0.704	0.747	0.746	0.845	0.494	0.494	0.676	0.535	0.535
BERT + Mean + LN +KM	0.804	0.538	0.537	0.356	0.213	0.212	0.731	0.747	0.746	0.845	0.494	0.494	0.676	0.535	0.535
BERT + Max + LN + EFCM	0.778	0.497	0.497	0.402	0.217	0.162	-	-	-	-	-	-	-	-	-
DFTC	0.851	**0.626**	0.624	0.526	0.356	**0.352**	0.756	0.806	**0.795**	0.846	0.503	0.503	0.695	0.614	0.613
SDEC (0% MSE & 100% CSL)	0.775	0.44	0.42	0.475	0.35	0.248	0.744	0.772	0.776	0.842	0.495	0.468	0.632	0.52	0.46
SDEC (100% MSE & 0% CSL)	0.832	0.574	0.607	0.524	**0.393**	0.289	0.729	0.788	0.661	0.845	0.493	0.476	0.644	0.54	0.51
SDEC	**0.857**	0.625	**0.628**	**0.5363**	0.363	0.348	0.755	**0.808**	0.787	0.852	**0.506**	**0.504**	**0.704**	**0.627**	**0.616**

embeddings.

- **Mean** refers to the mean pooling strategy of the embeddings.
- **Max** refers to the max pooling strategy of the embeddings.
- **Last** refers to the last pooling strategy of the embeddings
- **KM** stands for K-means clustering algorithm.
- **EFCM** is Eigenspace-based Fuzzy C-means, a clustering method combining fuzzy logic and eigenspace transformations. [60]

In addition to neural-based methods, we evaluated traditional topic modeling approaches paired with clustering algorithms, such as *NMF + TF-IDF + K-Means* and *LSA + TF-IDF + Spherical K-Means*. As shown in Table III, these classical baselines performed worse than neural network-based models. This underscores the superiority of deep embedding approaches for capturing semantic relationships in high-dimensional text data.

Our SDEC system demonstrated consistent superiority, obtained a notable 85.7% accuracy, 62.5% NMI, and 62.8% ARI for the *AG News* dataset. This is noticeably better than the next best-performing model, DFTC, which achieved a marginally higher NMI of 62.6% but had slightly lower accuracy (85.1%) and ARI (62.4%). Several models, such as DEC and IDEC, on the other hand, trailed significantly, with accuracy levels below 81% and NMI values approximately 53.8%.

Our system's resilience is demonstrated by our SDEC approach's 53.63% accuracy, 36.3% NMI, and 34.8% ARI on the *Yahoo! Answers* dataset, which again outperforms the rest of the field. The next-best system for this dataset, DFTC, obtained similar but still lower accuracy of 52.6%, NMI of 35.6%, and ARI of 35.2%. Furthermore proving the consistency of our methodology in unsupervised clustering tasks, the models utilizing BERT with K-Means or DEC fared significantly worse, with accuracies ranging between 26% and 48.7%.

By achieving 75.5% accuracy, 80.8% NMI, and 78.7% ARI for the *DBPedia* dataset, SDEC once again demonstrated its resilience, closely matching the results of DFTC, which obtained slightly higher accuracy (75.6%) and ARI (79.5%) but a lower NMI (80.6%). With more clusters and a larger range of categories, this dataset exhibits a notable increase in complexity. Our strategy handled the increased complexity well, ensuring competitive results across all metrics, by means of optimized pooling, semantic loss, and layer normalization strategies.

The *Reuters 2* dataset, due to its simplicity, allowed multiple systems to perform well. Among the best performers, our SDEC had accuracy of 85.2%, NMI of 50.6%, and ARI of 50.4%. Nonetheless, the performance disparities between different systems were negligible because of the dataset's relative simplicity. Models such as BERT + Last + N + KM demonstrate this, achieving 85.6% accuracy and 49.1% ARI, proving that even more basic models can function well when the clustering problem is less complicated.

Lastly, on the more challenging *Reuters 5* dataset, our SDEC approach demonstrated superior performance with an accuracy of 70.4%, 62.7% NMI, and 61.6% ARI, significantly outperforming other systems such as DEC and IDEC, which struggled with accuracies below 38%. The better performance of SDEC on *Reuters 5* indicates that our framework also excels in dealing with moderately complex clustering tasks, where other unsupervised methods falter.

While SDEC and DFTC may achieve similar numerical results, SDEC offers important methodological benefits. Unlike DFTC's usage of static embeddings, SDEC fine-tunes representations during clustering using a dual-loss approach that jointly optimizes semantic alignment and reconstruction. This dynamic optimization and refinement make SDEC more robust and adaptable, especially for tasks needing deeper

semantic understanding.

To visually highlight the comparative performance among advanced neural-based clustering methods, we present Figure 5, which shows the clustering accuracy results specifically for deep learning-based techniques across all datasets. These figures (5a–5e) illustrate that the proposed SDEC framework consistently achieves strong performance compared to other deep clustering approaches, such as DEC, IDEC, and transformer-based K-means variants. Note that the figures exclude classical topic modeling-based methods and ablation study variants, which are comprehensively detailed numerically in Table III.

To further validate our proposed loss combination (MSE + CSL) and dynamic weight allocation, we performed targeted ablation experiments summarized in Table III. Specifically, we compared two variations of our framework: SDEC focusing exclusively on semantic alignment, where the loss emphasized angular similarity between embeddings (CSL only, 0% MSE and 100% CSL) and SDEC focusing exclusively on embedding regeneration, where the loss prioritized reconstructing the original embedding values (MSE only, 100% MSE and 0% CSL). The results from these experiments show that focusing solely on semantic alignment generally yields suboptimal performance across all datasets, as evidenced by decreased accuracy and ARI scores. In contrast, the embedding regeneration variant consistently outperformed semantic alignment alone, suggesting that emphasizing the faithful reconstruction of embedding values helps preserve more discriminative features for clustering. However, the full SDEC model, which dynamically balances both objectives, clearly surpasses both single-loss configurations, particularly on more complex datasets. These findings underscore the effectiveness and necessity of the integrated loss function design in the SDEC framework.

Table III provides a comprehensive summary of clustering performance results across all evaluated methods and datasets, including both classical topic modeling baselines, ablation variants of our proposed method, and state-of-the-art deep clustering frameworks. This extensive comparison clearly demonstrates that our proposed SDEC system consistently outperforms other unsupervised methods across all metrics. Whether the dataset complexity is high, moderate, or low, SDEC maintains superior accuracy, NMI, and ARI scores, highlighting its effectiveness and robustness across diverse clustering scenarios.

VII. Conclusions and Discussions

In order to improve text clustering accuracy, this research presented Semantic Deep Embedded Clustering (SDEC), a novel framework for unsupervised text clustering that combines semantic refinement with a combined semantic loss. With the use of transformer-based BERT embeddings and an optimized autoencoder, SDEC has proven to perform better on a range of datasets with varying degrees of complexity.

A. Summary of Findings

Our results show that SDEC performs consistently better than conventional clustering systems. Among the notable accomplishments are accuracy rates of 85.7% on *AG News* and 53.63% on the more complex *Yahoo! Answers*, which are higher than conventional approaches on these datasets. Additionally, SDEC demonstrated robust performance on *DBPedia*, *Reuters 2*, and *Reuters 5*, demonstrating the model's effectiveness in managing a variety of text corpora and obtaining good metrics on all datasets.

B. Key Insights

The SDEC framework's effectiveness stems from:

- The integration of semantic and distributional losses, ensuring deep semantic preservation within the clustering process.
- Semantic refinement techniques that enhance clustering definitions, particularly beneficial in complex datasets like *DBPedia* and *Yahoo! Answers*.
- Consistent performance across various datasets, illustrating the method's adaptability and robustness.

C. Contributions

SDEC's primary contributions include:

- An enhanced clustering framework that integrates semantic loss and semantic refinement for improved clustering accuracy and semantic integrity.
- Demonstrated improvements in clustering performance across five major datasets, showcasing versatility.
- Establishment of a robust methodology that surpasses traditional DEC and IDEC approaches, especially in complex clustering scenarios.

D. Limitations

Clustering is fundamentally an NP-hard problem [61], implying that no polynomial-time algorithm can guarantee a globally optimal solution. Therefore, the concept of a definitive, universally "correct" clustering outcome is inherently ambiguous in unsupervised learning, requiring reliance on heuristic and approximation-based methods. Although SDEC demonstrates strong performance, certain limitations remain. Firstly, the model assumes prior knowledge of the number of clusters, which might not always be realistic in open-domain or exploratory scenarios. Secondly, transformer-based embeddings, despite their proven effectiveness, incur substantial computational costs, potentially limiting the scalability of SDEC to extremely large datasets without specialized computational infrastructure or optimization. Thirdly, SDEC inherently lacks interpretability which is a common limitation of complex deep learning-based clustering models, making it challenging to directly explain why certain documents are grouped together, which may be a drawback in domains requiring transparency and explainability. Furthermore, the framework introduces numerous hyperparameters (e.g., loss weights, refinement thresholds, learning rates), and optimal tuning may require significant computational resources and domain expertise, potentially creating a barrier for new users. We also note that, the current experimental validation has been conducted exclusively on English-language datasets using BERT embeddings. While this setup is effective for the present benchmarks, the applicability of SDEC to multilingual contexts or with alternative embedding models remains

unexplored and is an important area for future research. Lastly, the SDEC framework depends on the initialization of cluster centers using the k-means++ strategy. While superior to random initialization, this approach remains stochastic and can influence final clustering outcomes. As a consequence, the clustering quality might vary across different runs, even with fixed hyperparameters, due to sensitivity in the initial cluster assignments within the latent space.

E. Future Directions

Potential areas for future research include:

- Exploring scalability to larger and more heterogeneous datasets to further test SDEC's effectiveness. To address scalability limitations arising from computationally expensive transformer-based embeddings, methods such as embedding compression, efficient retrieval strategies, or lightweight transformer models should be explored.
- Investigating hybrid models that merge unsupervised and supervised learning to refine feature representations and clustering outcomes. In particular, such hybrid approaches offer a promising path for addressing cluster imbalance [62], [63], as supervised guidance and imbalance-aware loss functions can help ensure that minority classes are better represented and more accurately clustered.
- Developing robust initialization methods or ensemble clustering approaches to mitigate sensitivity issues related to stochastic initialization of cluster centers. Advanced initialization strategies, meta-learning, or consensus clustering methods could provide more stable and consistent clustering performance.
- Extending SDEC applications to real-world tasks such as document categorization and recommendation systems, to validate its practical utility. Evaluating performance in realistic scenarios will inform improvements in computational efficiency, interpretability, and practical robustness.
- Addressing the challenge of multimodal data, where text is combined with other modalities such as images, audio, or structured metadata. Adapting SDEC to jointly process and align heterogeneous feature spaces will require extending the framework to incorporate modality-specific encoders and cross-modal fusion mechanisms, enabling more comprehensive clustering of complex, real-world datasets.

Overall, SDEC sets a new standard for unsupervised text clustering, effectively addressing both the challenge of semantic preservation and the need for high clustering accuracy. Future enhancements and applications of this framework have the potential to significantly advance the field of text clustering.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.

[2] A. Agrawal, S. Tripathi, M. Vardhan, V. Sihag, G. Choudhary, and N. Dragoni, "Bert-based transfer-learning approach for nested named-entity recognition using joint labeling," *Applied Sciences*, vol. 12, no. 3, p. 976, 2022.

[3] C. Sun, X. Qiu, Y. Xu, and X. Huang, "How to fine-tune bert for text classification?" in *Chinese computational linguistics: 18th China national conference, CCL 2019, Kunming, China, October 18–20, 2019, proceedings 18*. Springer, 2019, pp. 194–206.

[4] M. W. U. Rahman, S. Shao, P. Satam, S. Hariri, C. Padilla, Z. Taylor, and C. Nevarez, "A bert-based deep learning approach for reputation analysis in social media," in *2022 IEEE/ACS 19th International Conference on Computer Systems and Applications (AICCSA)*. IEEE, 2022, pp. 1–8.

[5] B. Wolff, E. Seidlmayer, and K. U. Förstner, "Enriched bert embeddings for scholarly publication classification," in *International Workshop on Natural Scientific Language Processing and Research Knowledge Graphs*. Springer, 2024, pp. 234–243.

[6] M. W. Ur Rahman, M. M. Abrar, H. G. Copening, S. Hariri, S. Shao, P. Satam, and S. Salehi, "Quantized transformer language model implementations on edge devices," in *2023 International Conference on Machine Learning and Applications (ICMLA)*, 2023, pp. 709–716.

[7] S. Rajpal and R. Usbeck, *BERTologyNavigator: Advanced Question Answering with BERT-based Semantics*, ser. CEUR Workshop Proceedings. CEUR-WS.org, 2023, vol. 3592.

[8] M. Zaib, D. H. Tran, S. Sagar, A. Mahmood, W. E. Zhang, and Q. Z. Sheng, "Bert-coqac: Bert-based conversational question answering in context," in *Parallel Architectures, Algorithms and Programming: 11th International Symposium, PAAP 2020, Shenzhen, China, December 28–30, 2020, Proceedings 11*. Springer, 2021, pp. 47–57.

[9] A. Kel'manov and V. Khandeev, "Fast and exact algorithms for some np-hard 2-clustering problems in the one-dimensional case," in *International Conference on Analysis of Images, Social Networks and Texts*. Springer, 2019, pp. 377–387.

[10] T. F. Gonzalez, "On the computational complexity of clustering and related problems," in *System Modeling and Optimization: Proceedings of the 10th IFIP Conference New York City, USA, August 31–September 4, 1981*. Springer, 1982, pp. 174–182.

[11] R. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.

[12] T. Mikolov, K. Chen, G. S. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *International Conference on Learning Representations*, 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:5959482

[13] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman, "Indexing by latent semantic analysis," *Journal of the American Society for Information Science*, vol. 41, no. 6, pp. 391–407, 1990.

[14] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[15] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Advances in neural information processing systems*, vol. 26, 2013.

[16] S. P. Lloyd, "Least squares quantization in pcm," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[17] F. Murtagh and P. Contreras, "Algorithms for hierarchical clustering: an overview," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 2, no. 1, pp. 86–97, 2012.

[18] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *KDD*, vol. 96, 1996, pp. 226–231.

[19] R. J. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Pacific-Asia conference on knowledge discovery and data mining*. Springer, 2013, pp. 160–172.

[20] I.-M. Rădulescu, A. Boicea, C.-O. Truică, E.-S. Apostol, M. Mocanu, and F. Rădulescu, "Denlac: Density levels aggregation clustering–a flexible clustering method–," in *International Conference on Computational Science*. Springer, 2021, pp. 316–329.

[21] A. Ng, M. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," *Advances in neural information processing systems*, vol. 14, 2001.

[22] I. Radulescu, C. Truica, E. Apostol, A. Boicea, M. Mocanu, D. Popeanga, and F. Radulescu, "Density-based text clustering using document embeddings," in *Proceedings of the 36th IBIMA Conference, Granada, Spain*, 2020, pp. 4–5.

[23] R.-G. Radu, I.-M. Rădulescu, C.-O. Truică, E.-S. Apostol, and M. Mocanu, "Clustering documents using the document to vector model for dimensionality reduction," in *2020 ieee international conference on automation, quality and testing, robotics (aqtr)*. IEEE, 2020, pp. 1–6.

[24] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *Physics reports*, vol. 659, pp. 1–44, 2016.

[25] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, 2008.
[26] I.-M. Rădulescu, C.-O. Truică, E.-S. Apostol, and C. Dobre, "Enhancing scientific collaborations using community detection and document clustering," in *2020 IEEE 16th International Conference on Intelligent Computer Communication and Processing (ICCP)*. IEEE, 2020, pp. 43–50.
[27] E.-S. Apostol, C.-O. Truică, and A. Paschke, "Contcommrtd: A distributed content-based misinformation-aware community detection system for real-time disaster reporting," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
[28] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *International Conference on Machine Learning*, 2016, pp. 478–487.
[29] K. Ghasedi Dizaji, A. Herandi, C. Deng, W. Cai, and H. Huang, "Deep clustering via joint convolutional autoencoder embedding and relative entropy minimization," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 5736–5745.
[30] F. Li, H. Qiao, and B. Zhang, "Discriminatively boosted image clustering with fully convolutional auto-encoders," *Pattern Recognition*, vol. 83, pp. 161–173, 2018.
[31] X. Guo, X. Liu, E. Zhu, and J. Yin, "Improved deep embedded clustering with local structure preservation," in *IJCAI*, vol. 17, 2017, pp. 1753–1759.
[32] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
[33] N. Dilokthanakul, P. A. Mediano, M. Garnelo, M. C. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan, "Deep unsupervised clustering with gaussian mixture variational autoencoders," *arXiv preprint arXiv:1611.02648*, 2016.
[34] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
[35] S. K. Mukherjee, H. Asnani, E. Lin, and S. Kannan, "Clustergan: Latent space clustering in generative adversarial networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 4610–4617.
[36] Y. Lee, C. Park, and S. Kang, "Deep embedded clustering framework for mixed data," *IEEE Access*, vol. 11, pp. 33–40, 2022.
[37] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *Journal of machine Learning research*, vol. 3, no. Jan, pp. 993–1022, 2003.
[38] D. Lee and H. S. Seung, "Algorithms for non-negative matrix factorization," *Advances in neural information processing systems*, vol. 13, 2000.
[39] M. Mitroi, C.-O. Truică, E.-S. Apostol, and A. M. Florea, "Sentiment analysis using topic-document embeddings," in *2020 IEEE 16th international conference on intelligent computer communication and processing (ICCP)*. IEEE, 2020, pp. 75–82.
[40] A. Petrescu, C.-O. Truică, and E.-S. Apostol, "Sentiment analysis of events in social media," in *2019 IEEE 15th International Conference on Intelligent Computer Communication and Processing (ICCP)*. IEEE, 2019, pp. 143–149.
[41] A. Petrescu, C. Truică, E. Apostol, and A. Paschke, "Edsa-ensemble: An event detection sentiment analysis ensemble architecture." *arXiv preprint arXiv:2301.12805*, 2023.
[42] C.-O. Truică, E.-S. Apostol, T. Stefu, and P. Karras, "A deep learning architecture for audience interest prediction of news topic on social media," in *International Conference on Extending Database Technology (EDBT2021)*, 2021, pp. 588–599.
[43] C.-O. Truica, F. Radulescu, and A. Boicea, "Comparing different term weighting schemas for topic modeling," in *2016 18th international symposium on symbolic and numeric algorithms for scientific computing (SYNASC)*. IEEE, 2016, pp. 307–310.
[44] C.-O. Truica, E. S. Apostol, and C. A. Leordeanu, "Topic modeling using contextual cues," in *2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*. IEEE, 2017, pp. 203–210.
[45] C.-O. Truică, E.-S. Apostol, M.-L. Șerban, and A. Paschke, "Topic-based document-level sentiment analysis using contextual cues," *Mathematics*, vol. 9, no. 21, p. 2722, 2021.
[46] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1532–1543.
[47] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
[48] A. Subakti, H. Murfi, and N. Hariadi, "The performance of bert as data representation of text clustering," *Journal of big Data*, vol. 9, no. 1, p. 15, 2022.
[49] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, "Self-normalizing neural networks," *Advances in neural information processing systems*, vol. 30, 2017.
[50] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 11, pp. 2579–2605, 2008.
[51] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
[52] P. J. Rousseeuw, "Silhouettes: a graphical aid to the interpretation and validation of cluster analysis," *Journal of computational and applied mathematics*, vol. 20, pp. 53–65, 1987.
[53] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
[54] Yahoo, "Yahoo! answers dataset for text classification," Yahoo Webscope Program, 2008, subset used for natural language processing tasks, containing 140,000 samples. [Online]. Available: http://webscope.sandbox.yahoo.com
[55] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, "Dbpedia: A nucleus for a web of open data," in *International Semantic Web Conference*. Springer, 2007, pp. 722–735.
[56] Carnegie Group, Inc. and Reuters, Ltd., "Reuters-21578 text categorization collection," 1987, accessed: 2023-09-23. [Online]. Available: https://archive.ics.uci.edu/dataset/137/reuters+21578+text+categorization+collection
[57] A. Strehl and J. Ghosh, "Cluster ensembles - a knowledge reuse framework for combining multiple partitions," *Journal of Machine Learning Research*, vol. 3, pp. 583–617, 2002.
[58] R. Guan, H. Zhang, Y. Liang, F. Giunchiglia, L. Huang, and X. Feng, "Deep feature-based text clustering and its explanation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 8, pp. 3669–3680, 2020.
[59] A. F. Agarap, "Text classification and clustering with annealing soft nearest neighbor loss," *arXiv preprint arXiv:2107.14597*, 2021.
[60] T. Muliawati and H. Murfi, "Eigenspace-based fuzzy c-means for sensing trending topics in twitter," in *AIP Conference Proceedings*, vol. 1862, no. 1. AIP Publishing, 2017.
[61] S. Dasgupta, "The hardness of k-means clustering," *UC San Diego Technical Reports*, 2008.
[62] W.-C. Lin, C.-F. Tsai, Y.-H. Hu, and J.-S. Jhang, "Clustering-based undersampling in class-imbalanced data," *Information Sciences*, vol. 409, pp. 17–26, 2017.
[63] C.-O. Truica and C. A. Leordeanu, "Classification of an imbalanced data set using decision tree algorithms," *Univ. Politech. Bucharest Sci. Bull. Ser. C Electr. Eng. Comput. Sci*, vol. 79, pp. 69–84, 2017.

BIOGRAPHY

Mohammad Wali Ur Rahman graduated with a Master of Science degree in Electrical and Computer Engineering from the University of Arizona in 2023 and is currently pursuing his Ph.D. in the same field at the same institution. He has been engaged as a Research Assistant at the Autonomic Computing Lab, a branch of the NSF-CAC (NSF Center for Autonomic Computing). His academic and research pursuits are centered around Text Mining & Analysis, Natural Language Processing, Machine Learning, Neural Networks, Artificial Intelligence and cybersecurity.

Ric Nevarez is the Chief Technology Officer at Trust Web. He graduated with a Bachelor of Science degree in Nutritional Biochemistry from Brigham Young University in 2014. With over 14 years of experience in the artificial intelligence and natural language processing industry, Ric Nevarez has been a strong advocate for sustainable and ethical AI practices. As an experienced product manager and entrepreneur, he has co-founded several successful AI startups, securing substantial funding and developing innovative AI products.

Lamia Tasnim Mim is a graduate assistant at New Mexico State University, where she earned her Master's in Computer Science (2023). She specializes in Artificial Intelligence, Machine Learning, and Natural Language Processing, with a focus on developing robust and scalable deep learning models. Lamia served as a Machine Learning Engineer at Avirtek, Inc., where she developed streaming data pipelines, deployed ML models, and enhanced data recovery accuracy through advanced techniques.

Dr Salim Hariri (Senior Member, IEEE) received an M.Sc. degree from Ohio State University in 1982, and a Ph.D. degree in Computer Engineering from the University of Southern California in 1986. He is a Professor in the Department of Electrical and Computer Engineering, the University of Arizona, and the Director of the NSF Center for Cloud and Autonomic Computing (NSF-CAC). His research focuses on autonomic computing, artificial intelligence, machine learning, cybersecurity, cyber resilience, and cloud security.

APPENDIX A
PREPROCESSING AND LEMMATIZATION

The preprocessing step is critical in ensuring that the textual data is clean, consistent, and suitable for further analysis. This section describes the preprocessing steps done on the text data: lemmatizing the tokens, cleaning the text with regular expressions, removing URLs, and expanding contractions. The aim is to convert the unprocessed text into a format that is standardized and suitable for producing embeddings.

To start, for greater consistency and clarity, the text data frequently needs to have contractions expanded. We enlarge them in the text by using a predefined dictionary of contractions.

Next, using a regular expression pattern, we preserve only alphabets and choose punctuation characters (like '!', '?', ',', and '.'). The text is made cleaner by removing all other characters.

To highlight the important passages, certain redundant words are also eliminated from the text. Following this, the text has been cleaned and tokenized into individual words, dividing the text into manageable chunks for subsequent processing.

After tokenization, more filtering is used to eliminate particular undesirable tokens that don't improve the text analysis. After that, the tokens are lemmatized to return to their base or root form, which aids in normalizing the text and simplifying the vocabulary.

After lemmatization, the normalized text is represented by joining the tokens back into a single string. These preprocessing procedures help us make sure the text data is clean, consistent, and suitable for producing high-quality embeddings. Our clustering framework's later steps are built upon this processed text data.

APPENDIX B
BERT EMBEDDING GENERATION

We use BERT (Bidirectional Encoder Representations from Transformers) embeddings, namely the pre-trained "bert-large-cased" model, to take full advantage of the power of transformer models for text clustering. For the text data to be successfully converted into high-quality embeddings that accurately capture the text's semantic meaning, a number of steps must be taken during the generation of BERT embeddings.

The key steps for generating BERT embeddings are as follows:

- **Load BERT Tokenizer and Model**:
  - Utilize the `transformers` library to load the BERT tokenizer and model.
  - The tokenizer converts text into tokens that the BERT model can process.
- **Encode Text Data**:
  - For each text sample, the tokenizer encodes the text, adding special tokens and ensuring proper padding and truncation to maintain a consistent input length.
  - The encoded inputs are passed through the BERT model to obtain token embeddings from the last hidden state.
- **Pooling Strategies**:
  - We experimented with three different pooling strategies to derive fixed-size representations for each text sample:
    - **CLS Pooling**: Taking the [CLS] token embedding, designed to capture the aggregate representation of the input text.
    - **Mean Pooling**: Calculating the average of the token embeddings to obtain a representation that reflects the overall content of the text. This can be represented as:

$$h[k] = \frac{1}{n}\sum_{i=1}^{n} h_{ik}, \tag{20}$$

      where $h[k]$ is the k-th entry of the feature-extracted representation vector h, and h_{ik} is the k-th entry of the i-th token embedding.
    - **Max Pooling**: Taking the maximum value across all token embeddings, which helps to capture the most salient features of the text. This can be represented as:

$$h[k] = \max_{1\leq i\leq n} h_{ik}, \tag{21}$$

      where $h[k]$ is the k-th entry of the feature-extracted representation vector h.
  - Among these strategies, mean pooling typically yields the best results for clustering, providing a balanced representation of the text content.
- **Convert to Numpy Arrays**:
  - Convert the mean-pooled BERT embeddings into numpy arrays for further processing.
- **Normalization**:
  - We experimented with two types of normalization techniques:
    - **Standard Normalization**: Applying a `StandardScaler` to ensure the embeddings are on a comparable scale. This transforms the vector representation into a vector with a norm of 1. This can be represented as:

$$\bar{h_i} = \frac{h_i}{\|h_i\|}, \tag{22}$$

      where $\bar{h_i}$ is the normalized vector.
    - **Layer Normalization**: Ensuring the stability and consistency of the embeddings across different layers by avoiding covariate shift problems. This can be represented as:

$$\bar{h_i} = \frac{h_i - \phi_i}{\sigma_i}, \tag{23}$$

      where ϕ_i and σ_i are the mean and standard deviation of the feature vector h_i, respectively.
  - Fit and transform the training embeddings, and transform the testing embeddings using the same scaler.

By following these steps, we generate BERT embeddings that preserve the semantic meaning of the text and are standardized for subsequent use in the autoencoder and clustering

TABLE IV: Comparison of Key Hyperparameters Across DEC, IDEC, and SDEC

Category	DEC	IDEC	SDEC (Ours)
Architecture	d–500–500–2000–10	Same + mirrored decoder	d–2048–1024–512–256–128, symmetric decoder
Activation Function	Not stated	ReLU	SeLU, linear at output
Regularization	None specified	None specified	L2 regularization
Embedding Dimension	10	10	128
Batch Size	256	256	16 (AE), 32 (Clustering)
Autoencoder Optimizer	SGD	Adam/SGD	Adam
AE Learning Rate	0.1	Varied: 0.1 to 1×10^{-4}	Varied: 1×10^{-3} to 1×10^{-5}
Epochs	100,000 iterations	Not specified	100–1500 epochs (AE), 20,000 iterations (Clustering)
K-Means Restarts	20	20	20 - 2000 (KMeans++)
Clustering Loss	KL divergence	KL + Reconstruction	KL + Semantic Reconstruction
Clustering Optimizer	SGD (0.1)	Adam/SGD	SGD (0.01, momentum 0.9)
Clustering Batch Size	256	256	32
KL α (Student's t-kernel)	Implicit	Not stated	Explicit α = 1.0
KL Convergence Threshold	δ = 0.001	δ = 0.001	δ = 0.001
Update Interval	Not stated	Not stated	varied between 2, 5, 10, 50 & 100 iterations
Cluster Coefficient	N/A	0.1	0.1
Cluster Refinement Threshold	N/A	N/A	λ threshold $\in$ [0.1, 0.7]

framework. These embeddings form the foundation for the next stages of our methodology, ensuring high-quality input data for effective clustering.

APPENDIX C
HYPERPARAMETERS AND EXPERIMENT SETTINGS

This appendix provides a comprehensive overview of the hyperparameters and experimental configurations employed for training and evaluating the proposed Semantic Deep Embedded Clustering (SDEC) model. To ensure clarity and ease of comparison with prior models, a summary of key hyperparameter settings from the DEC and IDEC models is also included (Table IV). Additionally, sensitivity analyses conducted specifically for the critical hyperparameters unique to the SDEC framework are discussed.

A. Comparative Hyperparameter Overview

Table IV summarizes the principal hyperparameter choices adopted in DEC, IDEC, and our proposed SDEC framework. In alignment with established practice in deep clustering literature, nearly all hyperparameters in SDEC are selected from commonly used values in deep neural network training and transformer-based (e.g., BERT) embedding workflows. Consistent with DEC and IDEC, we deliberately avoid dataset-specific hyperparameter tuning to promote generalizability and fair comparison. Nevertheless, SDEC introduces several additional design elements, such as enhanced regularization and loss weighting strategies, which are reflected in the table and further explored through targeted sensitivity analyses.

B. Autoencoder Hyperparameters

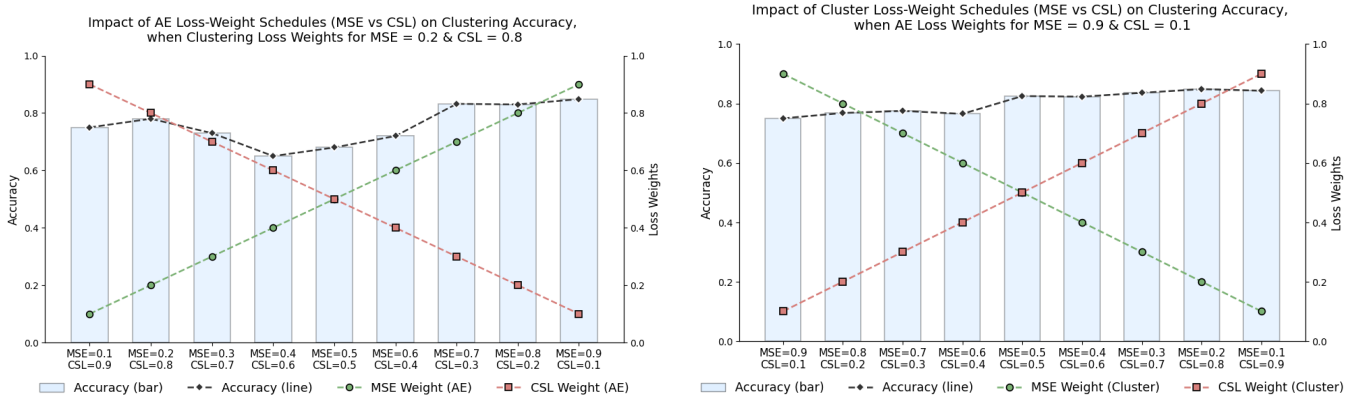
The autoencoder architecture plays a pivotal role in our experiments. It employs L2 regularization and SeLU activation functions throughout its hidden layers. The encoder comprises four fully connected layers with 2048, 1024, 512, and 256 units, respectively, followed by a bottleneck (latent) layer of 128 units to capture the key features of the input embeddings. The decoder mirrors the encoder's architecture with layers of 256, 512, 1024, and 2048 units, and the output layer uses a linear activation function for reconstruction. Training of the autoencoder is performed using the Adam optimizer, with a learning rate varied between 1×10^{-3} and 1×10^{-5}, a batch size of 16, and a composite reconstruction loss combining Mean Squared Error (MSE) and Cosine Similarity Loss (CSL). The number of training epochs ranges from 100 to 1500, depending on the dataset size and complexity.

C. Clustering and Fine-Tuning Hyperparameters

Cluster centroids are initialized using the K-Means++ algorithm, with initialization runs ranging from 2×10^1 to 2×10^3. Given that clustering outcomes from K-Means-based methods are highly sensitive to centroid initialization, employing a higher number of initializations typically yields more robust and stable cluster configurations, providing an improved foundation for subsequent optimization steps. For clustering optimization, we employ stochastic gradient descent (SGD) with a learning rate of 0.01 and momentum of 0.9, a widely adopted setting that provides effective convergence and avoids local minima. Following the recommendations established by the authors of IDEC, the clustering coefficient was set to 0.1 to suitably balance the KL divergence and semantic reconstruction objectives, facilitating the generation of semantically coherent clusters. The clustering kernel employs a Student's t-distribution with an explicit parameter α set to 1.0, enabling the clustering process to effectively manage varying cluster densities and outliers in the embedding space. The convergence criteria were defined through a KL divergence threshold (δ) of 0.001 and a maximum delta-label tolerance of 0.001, ensuring that the algorithm halts only upon achieving substantial stability in cluster assignments. During the clustering phase, cluster assignments and target distributions were updated at intervals experimentally varied among 2, 5, 10, 50, and 100 iterations, with the maximum number of clustering iterations capped at 20,000 using a batch size of 32.

D. Loss Weight Sensitivity Experiments

Although the SDEC algorithm employs dynamically allocated reconstruction loss weights, controlled experiments were conducted to explicitly examine how fixed variations in the



(a) AE-phase loss-weight variations (MSE vs CSL) (b) Clustering-phase loss-weight variations (MSE vs CSL)

Fig. 6: Impact of loss-weight scheduling on clustering accuracy: (a) Autoencoder (AE) phase and (b) clustering phase.

Mean Squared Error (MSE) and Cosine Similarity Loss (CSL) weights influence clustering performance. These experiments involved independent runs on the AG News dataset, without targeting optimal performance explicitly, but rather aiming to investigate general sensitivity trends in clustering accuracy.

- **Autoencoder Phase (AE) Loss Weight Sweep:** During the autoencoder pretraining phase, we systematically varied the loss weight ratio between MSE and CSL while maintaining fixed weights (MSE=0.2, CSL=0.8) during the clustering phase. Results (Figure 6a) revealed that clustering accuracy notably declines when MSE and CSL weights are balanced or nearly balanced (e.g., 0.5–0.5 or 0.6–0.4). A plausible explanation for this accuracy dip is that balanced weighting may cause conflicting objectives between accurate embedding reconstruction (MSE) and semantic alignment (CSL), reducing the clarity and discriminative capability of the learned embeddings. Conversely, when one loss dominates, particularly the MSE, the autoencoder tends to learn more stable and consistent latent representations, leading to higher accuracy.
- **Clustering Phase Loss Weight Sweep:** For this experiment, the autoencoder training weights were held constant (MSE=0.9, CSL=0.1), emphasizing embedding stability. Clustering phase loss weights, however, were varied. Results (Figure 6b) clearly indicated improved clustering accuracy as the CSL weight increased. This outcome highlights the importance of semantic alignment in the clustering stage, where a stronger CSL emphasis encourages cluster centroids to align more meaningfully with semantically similar groups of embeddings, thereby improving overall cluster quality.

The rationale behind setting constant weights of MSE=0.2 and CSL=0.8 in the clustering phase (during the AE-phase weight variations) was to prioritize semantic similarity during clustering, allowing us to assess whether stable embeddings from the autoencoder training phase influenced clustering effectiveness. Conversely, we chose fixed autoencoder-phase weights (MSE=0.9, CSL=0.1) during the clustering-phase variations to maintain consistent and stable initial embeddings, enabling a clear analysis of how varying clustering weights impact clustering performance.

These analyses underscore the robustness of the dynamic weighting strategy originally adopted by the SDEC framework, demonstrating clear insights into the individual and combined contributions of reconstruction losses during autoencoder training and clustering.

E. Cluster Refinement Hyperparameters

The semantic refinement step introduces a refinement threshold (λ), which specifies the minimum required increase in semantic similarity for data points to be reassigned to new clusters. The main goal of this sensitivity analysis was to thoroughly understand how varying the λ threshold influences the clustering performance, rather than achieving the optimal clustering outcome itself. To accomplish this, we conducted systematic experiments using randomly initialized models across four benchmark datasets: *AG News*, *Yahoo! Answers*, *Reuters 5*, and *DBPedia*. These experiments covered a comprehensive range of λ values from 0.1 to 0.9, evaluated in increments of 0.01, as illustrated in Figure 7.

Our results consistently demonstrate that moderate λ values (approximately between 0.1 and 0.7) yield the best clustering outcomes. The rationale behind this finding relates directly to the nature of high-dimensional semantic embeddings, such as those generated by transformer-based models like BERT. Although initial clustering methods (e.g., K-means++) can effectively initialize clusters, they may not fully capture subtle semantic distinctions due to inherent limitations of centroid-based methods in high-dimensional feature spaces. The semantic refinement step addresses this issue by making minor adjustments to cluster assignments based on nuanced semantic similarity, thereby improving the semantic coherence within clusters.

If the refinement threshold λ is set too low (below 0.1), the model tends to make aggressive, often unnecessary reassignments, disrupting cluster stability and coherence. Conversely, excessively high values of λ (above 0.7) severely restrict the model's ability to make meaningful refinements, as the threshold for reassignment becomes overly stringent. Such

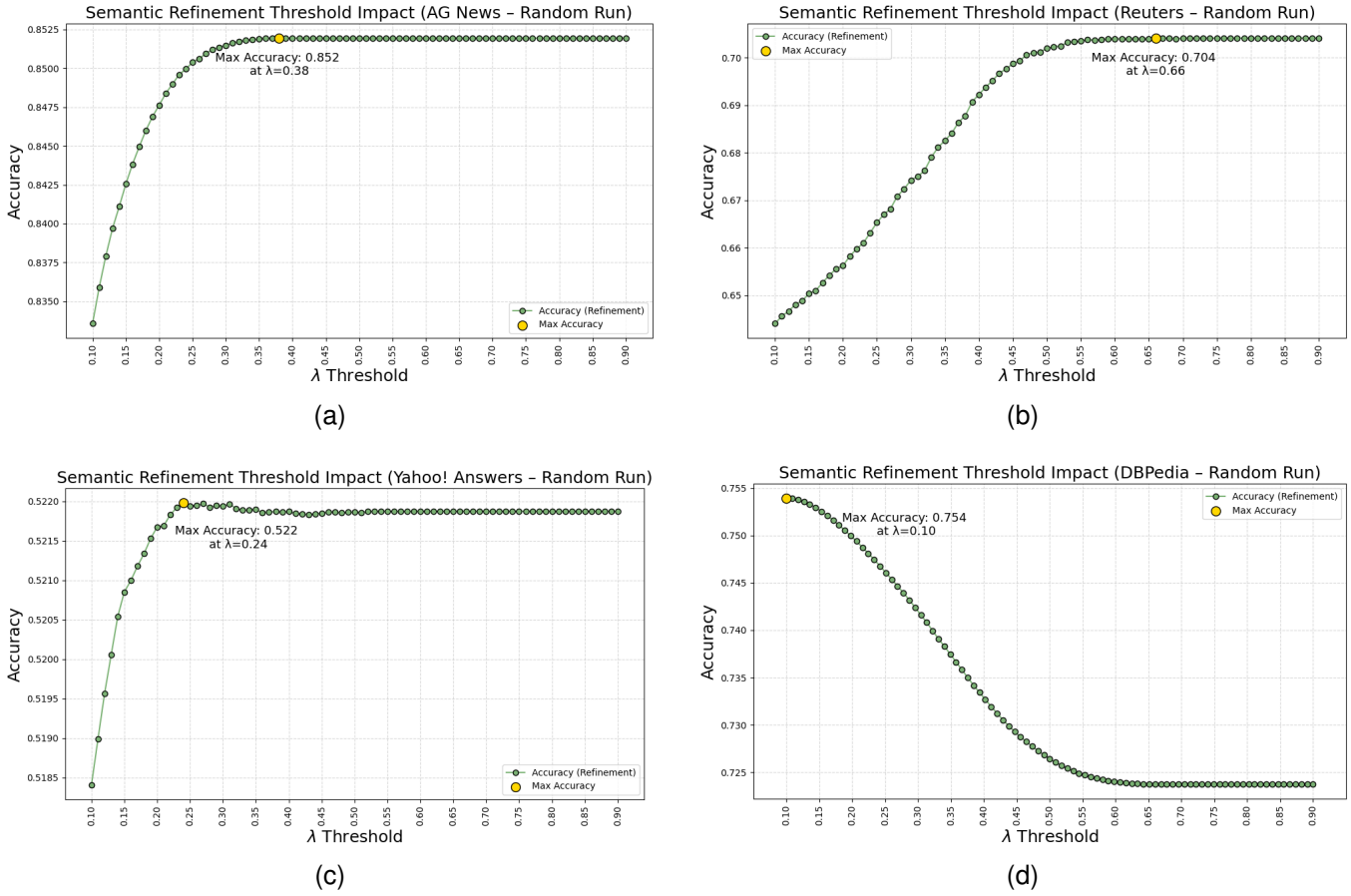


Fig. 7: Sensitivity analysis of the semantic refinement threshold λ on (a) AG News, (b) Reuters 5, (c) Yahoo! Answers, and (d) DBPedia datasets.

conservatism reduces the potential improvements in cluster quality, especially when the initial clustering quality leaves room for semantic enhancement.

Hence, based on these empirical observations, we recommend selecting a refinement threshold (λ) within the range of 0.1 to 0.7 as a sensible starting point. Since the ideal setting of this hyperparameter depends on the specific semantic and structural characteristics of the dataset at hand, researchers and practitioners should consider λ a tunable hyperparameter. The refinement threshold must therefore be selected with careful consideration of the dataset's complexity, semantic characteristics, and the performance expectations of the clustering system.

Ultimately, the semantic refinement process significantly enhances the quality of clusters, provided the threshold λ is carefully tuned to balance sensitivity and specificity in cluster reassignments. This controlled adjustment ensures robust and meaningful semantic alignment, thereby improving overall cluster coherence and reliability across diverse textual datasets.